

Lab 06 — Stacks, Brackets and Postfix

DSA27 Lab Manual · Week 6 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 6

Contents

Part 0 — Before you start	2
Part 1 — push, pop and peek: the top at the end	4
Part 2 — Measure it: the dishonest stack	6
Part 3 — <code>is_balanced(text)</code>	8
Part 4 — <code>evaluate_postfix(tokens)</code> , in <code>dsa/translation.py</code>	10
Part 5 — <code>infix_to_postfix(tokens)</code> : the shunting-yard algorithm	12
Part 6 — The whole calculator	14
Part 7 — Exercises at a glance	15
Part 8 — Take-home practice (not graded)	16
Part 9 — Bridge to next week's lecture: queues	16
Summary	17
Answers to the checkpoints	18

How to use this lab. The routine from Week 4 on: read the section of Lecture 06 that each part names, **draw before you code** — on paper, with the stack as a column and the top at the end — and predict at each **Checkpoint** before you run anything (answers at the end). Then build one method at a time and run the named tests after each part. The stack itself is the smallest class you will write this term; the work of this lab is the three algorithms that stand on it.

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>dsa/stack.py</code> — <code>Stack.push</code> , <code>Stack.pop</code> , <code>Stack.peek</code> , <code>is_balanced</code> , <code>infix_to_postfix</code> ; and <code>evaluate_postfix</code> in <code>dsa/translation.py</code>

Graded by	tests/test_stack_queue.py -k "lifo or peek or empty_behaviour or balanced or infix" (12 tests) and tests/test_translation.py -k evaluate_postfix (11 tests)
Connects to	Lecture 06 — Stacks; Lecture 04 (your DynamicArray); Lecture 07 — Queues

What you will be able to do

1. implement the Stack ADT on your own DynamicArray, with every operation $O(1)$, and say why the top must be the **end** of the array;
 2. measure a stack whose top is index 0, and explain why it passes every test and is still wrong;
 3. check balanced brackets with a stack, and name the three ways a text can be unbalanced;
 4. evaluate a postfix expression with one stack, getting the operand order right;
 5. trace the shunting-yard algorithm on paper, with an output list and an operator stack, and then implement it;
 6. chain the two into a calculator: infix, to postfix, to a value;
 7. read a failing stack test and go straight to the line that caused it.
-

Part 0 — Before you start

0.1 Environment and prerequisites

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
pytest -m "not challenge" -q          # the environment check — must pass
pytest tests/test_dynamic_array.py -q # your Week 4 work — must pass: 19 tests
```

The second line is not optional. Your Stack keeps its items in **your own** DynamicArray from Week 4 — the storage rule of Lecture 02 at work — so a bug in last week's `append`, `pop` or `__getitem__` shows up this week as a stack bug. If `DynamicArray.append` still raises `NotImplementedError`, every stack test that pushes fails with `NotImplementedError`, and the last line of the traceback is in `dsa/dynamic_array.py`, not in `dsa/stack.py`. Fix Week 4 first.

0.2 The right `-k` filter

`tests/test_stack_queue.py` holds the tests for this week **and** next week's queues. `pytest -k` matches the words you give it against each test's name **and the name of its file** — and the file is called `test_stack_queue.py`. So `-k stack` selects all 18 tests, queues included — and so does any filter with `stack` or `queue` in it. The six queue tests then fail with `NotImplementedError` until Week 7, which is noise, not your bug. Use the names of the tests themselves, as Lecture 06 does:

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or empty_behaviour or balanced or infix"
```

That selects exactly the 12 stack tests (6 deselected):

Filter word	Tests it selects
lifo	test_stack_is_lifo
peek	test_stack_peek_does_not_remove
empty_behaviour	test_stack_empty_behaviour
balanced	test_is_balanced — 7 cases
infix	test_infix_to_postfix and test_infix_to_postfix_is_left_associative

The same trap waits in tests/test_translation.py; Part 4 deals with it.

0.3 Read the skeleton

Open dsa/stack.py. Read the module docstring first: it tells you where the top goes before you write a line.

Given to you	Yours
<code>__init__(values=())</code> — makes <code>self._items = DynamicArray()</code> and pushes each value	<code>push(value)</code> — $O(1)$ amortised
<code>is_empty()</code> , <code>__len__()</code> — both ask <code>self._items</code>	<code>pop()</code> — $O(1)$; <code>IndexError</code> when empty
<code>__repr__()</code> — prints the items bottom to top, and the top	<code>peek()</code> — $O(1)$; <code>IndexError</code> when empty
	<code>is_balanced(text)</code> — $O(n)$
	<code>infix_to_postfix(tokens)</code> — $O(n)$

And in dsa/translation.py, **only** `evaluate_postfix(tokens)` is this week's. Everything else in that file — `tokenize`, `Parser`, `evaluate`, `calculate` — is Week 15. Leave it alone.

Look closely at `__repr__`:

```
return f"Stack({list(self._items)!r}) # top = {self._items[len(self._items) - 1]!r}"
```

It shows the items from bottom to top and names the **last** one as the top. That line is a contract: the top of this stack is the end of the array.

Checkpoint 1. On the untouched skeleton — before you have written anything — which of these lines work, and what do they give? Which raise, and why?

```
from dsa.stack import Stack
Stack()
len(Stack())
Stack().is_empty()
Stack([1, 2])
```

0.4 The storage rule, this week

- Stack stores its items in `self._items`, a `DynamicArray`. Not a Python list.
- `is_balanced`, `infix_to_postfix` and `evaluate_postfix` use **your Stack** as their working storage. A Python list used as a stack (append and pop) would pass the tests and break the rule — Lecture 06, “This Week”.
- Inputs and results are lists: `infix_to_postfix` takes a list of token strings and returns a list of token strings. That is the interface the tests use.

Part 1 — push, pop and peek: the top at the end

1.1 The idea

Lecture 06, “Two Honest Implementations”: the top goes where the $O(1)$ end is. On a dynamic array that is the **end**. `DynamicArray.append` is amortised $O(1)$ and `DynamicArray.pop()` with no argument removes the last item without moving anything else. So:

- **push** is an append;
- **pop** is a pop from the end — after checking the stack is not empty;
- **peek** reads the last item — after the same check — and removes nothing.

Every operation touches one slot. Nothing shifts.

1.2 Draw it

On paper, draw an array of capacity 4 and run `push(3)`, `push(1)`, `push(4)`, `pop()`, `push(5)`. After each step write down the size and which cell is the top. Then check your last picture with the course drawing helper:

```
from viz.draw import draw_array
draw_array([3, 1, 5], highlight=2, title="a stack on an array: the top is the last cell")
```

Notice what never happens in your drawing: no item ever moves to a new cell.

Checkpoint 2. What is printed?

```
s = Stack()
s.push('a'); s.push('b'); s.push('c')
print(s.pop())
s.push('d')
print(s.peak())
print(len(s))
print(s)
```

1.3 Write it

1. **push(value)**: append value to `self._items`. One line.
2. **pop()**: if the stack is empty, raise `IndexError('pop from empty stack')`. Otherwise remove the last item of `self._items` and return it.
3. **peek()**: if the stack is empty, raise `IndexError('peek at empty stack')`. Otherwise return the item at index `len(self._items) - 1`. Do not remove it.

Hints:

- Use `self.is_empty()` or `len(self._items) == 0` for the check — both are given and both are $O(1)$.
- Why raise your **own** `IndexError`, when `DynamicArray.pop` already raises one on an empty array? Checkpoint 3 answers that.
- Your methods talk to `self._items` only through its public methods: `append`, `pop`, `len`, indexing. Never reach into `self._items._block`.

Checkpoint 3. A student writes `peek` with no empty check at all:

```
def peek(self):
    return self._items[len(self._items) - 1]
```

What does `Stack().peek()` do with a correct Week 4 `DynamicArray`? Does `test_stack_empty_behaviour` pass? Why is it still worth writing the check?

1.4 Test it

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or empty_behaviour"
```

Three tests: `test_stack_is_lifo` (push 1, 2, 3; pop gives 3 then 2; length 1 left), `test_stack_peek_does_not_remove` (peek on `Stack([1, 2])` is 2, length still 2) and `test_stack_empty_behaviour` (`IndexError` from both pop and peek on an empty stack).

1.5 When it fails

The stack is a queue. push appends but pop calls `self._items.pop(0)`:

```
E assert 1 == 3
E + where 1 = pop()
E + where pop = Stack([2, 3]) # top = 3.pop
```

`test_stack_is_lifo` pushed 1, 2, 3 and got 1 back — the **first** in. The `__repr__` in the message says the top is 3, because the given `__repr__` trusts the contract that the top is the end. Push and pop must use the same end.

peek removes. peek written as `return self.pop()`:

```
E  assert 1 == 2
E    +  where 1 = len(Stack([1]) # top = 1)
```

The value was right (2); the length after it was not. Peek reads; it never changes the stack.

peek reads the wrong end. `return self._items[0]` gives the bottom:

```
E  assert 1 == 2
E    +  where 1 = peek()
E    +    where peek = Stack([1, 2]) # top = 2.peek
```

Off by one. `self._items[len(self._items)]` is one past the last item:

```
E  IndexError: 2
```

The last used index is `len - 1`. (If your `DynamicArray.__getitem__` supports negative indices, as Week 4 required, `self._items[-1]` also works.)

None instead of an error. `if self.is_empty(): return None`:

```
E  Failed: DID NOT RAISE IndexError
```

Lecture 06, “The Stack ADT”: a `None` can travel a long way through a program before anyone notices. The contract says `IndexError`.

Part 2 — Measure it: the dishonest stack

2.1 The idea

Lecture 06, “...and one dishonest one”: put the top at **index 0** of the array and every push shifts the whole stack one place right, every pop shifts it one place left. Each operation is $O(n)$, so n pushes followed by n pops are $O(n^2)$. The stack still behaves perfectly — same values, same order, same errors. Only the cost is wrong.

2.2 Draw it

Redraw Part 1.2 with the top at index 0: `push(3)`, `push(1)`, `push(4)`. Count how many items move at each push (0, then 1, then 2). With n items already on the stack, how many move for one more push?

```
draw_array([5, 1, 3], highlight=0, title="top at index 0: every push shifts the rest")
```

2.3 Build the dishonest stack and time both

Put this in a scratch file (not in dsa/) and run it once your Stack passes Part 1. It subclasses your Stack and changes only where the top is:

```
from dsa.stack import Stack
from viz.complexity import measure, plot_growth

class FrontStack(Stack):
    """The dishonest design: the top is index 0 of the DynamicArray."""

    def push(self, value):
        self._items.insert_at(0, value)          # shifts every item right

    def pop(self):
        if self.is_empty():
            raise IndexError('pop from empty stack')
        return self._items.pop(0)                # shifts every item left

    def peek(self):
        if self.is_empty():
            raise IndexError('peek at empty stack')
        return self._items[0]

def push_then_pop(cls):
    def work(n):
        s = cls()
        for i in range(n):
            s.push(i)
        while not s.is_empty():
            s.pop()
    return work

sizes = [200, 400, 800, 1600]
end = measure(push_then_pop(Stack), sizes, lambda n: n, repeat=1)
front = measure(push_then_pop(FrontStack), sizes, lambda n: n, repeat=1)
for n, a, b in zip(sizes, end[1], front[1]):
    print(f'{n:5d}  top at end {a * 1000:8.1f} ms  top at index 0 {b * 1000:8.1f} ms')
plot_growth({'top at index 0': front, 'top at the end': end},
            reference=['n^2'], loglog=True)
```

It takes around ten seconds, almost all of it in FrontStack. Do not add bigger sizes casually: at $n = 4,000$ the dishonest stack needs about half a minute per run on a typical laptop, while the

honest one needs a few hundredths of a second.

Checkpoint 4. Before you run it: (a) does `FrontStack` pass the three stack tests of Part 1? (b) When n doubles, by roughly how much should each column of times grow?

2.4 What you should see

- **Top at the end:** each doubling of n roughly doubles the time — linear in total, $O(1)$ per operation. At small sizes the numbers are noisy; look at the trend.
- **Top at index 0:** each doubling roughly **quadruples** the time. On the log-log plot its line runs parallel to the dashed $O(n^2)$ reference, and sits far above the honest line.
- The gap at $n = 1,600$ is a factor of a hundred or more, and it grows with n .

Your numbers will differ from your neighbour's; the shapes will not. You can check that `FrontStack` passes the tests by temporarily pasting its three methods over yours in `dsa/stack.py` and running Part 1's command — then undo it. That is the lesson of the lecture: **the tests check what your stack does. Only you can check how.** A TA will ask you what push costs.

Part 3 — `is_balanced(text)`

3.1 The idea

Lecture 06, "Application 1: Balanced Brackets". The rule "the most recently opened bracket must close first" is last in, first out — a stack. Read the text once, left to right:

- an **opener** `(`, `[` or `{`: push it;
- a **closer** `)`, `]` or `}`: the stack must be **non-empty**, and the opener you pop must be its **partner**;
- **anything else**: ignore it;
- at the **end of the text**: the stack must be **empty**.

3.2 Three checks, three ways to fail

Each check catches one way a text can be unbalanced. Leave any one out and six of the seven test cases still pass.

Input	What happens	The check that catches it
<code>([])</code>	<code>)</code> pops <code>[</code> — not its partner	the partner check
<code>)</code>	a closer arrives with nothing on the stack	the non-empty check
<code>(</code>	the text ends with <code>(</code> still on the stack	the empty-at-the-end check

3.3 Draw it

Trace `(a[b]{c})` on paper. After every character, write the stack (bottom on the left). You should find the letters change nothing and the stack is never taller than two:

```
(  ['(']
a  ['(']
[  ['(', '[']
b  ['(', '[']
]  ['(']
{  ['(', '{']
c  ['(', '{']
}  ['(']
)  []          <- empty at the end: balanced
```

Checkpoint 5. Predict `is_balanced` for each, and for each `False` name the check that decides it: `"a)(b"`, `"(]"`, `"((a)"`, `"{[(())]x"`, `"}{{"`.

3.4 Write it

1. Build a small dictionary mapping each closer to its opener: `)` to `(`, `]` to `[`, `}` to `{`. (A dictionary as a **lookup table** is fine; the storage rule is about the structure's working storage, which is the Stack.)
2. Make an empty Stack for the openers.
3. For each character: if it is an opener, push it. If it is a closer: if the stack is empty, return `False`; pop, and if what you popped is not the partner, return `False`.
4. After the loop, return whether the stack is empty.

The test compares with `is True` and `is False`, so return real booleans: `openers.is_empty()` is one.

Cost: every character is looked at once, and each push and pop is $O(1)$: $O(n)$ time, and $O(n)$ extra space in the worst case `((((...)`.

3.5 Test it

```
pytest tests/test_stack_queue.py -v -k balanced
```

Seven cases of `test_is_balanced`: `"", "()", "(a[b]{c})"` and `"no brackets at all"` are balanced; `"([)]"`, `"("` and `)"` are not — one for each row of the table in 3.2.

3.6 When it fails

No partner check — any closer just pops:

```
E AssertionError: assert True is False
E + where True = is_balanced('([)]')
```

No non-empty check — the closer pops an empty stack, and your own pop raises:

```
E   IndexError: pop from empty stack
```

The failing case is `test_is_balanced()-False`. An `IndexError` from inside `is_balanced` is never the right answer to “is this balanced?”.

No check at the end — return `True` after the loop:

```
E   AssertionError: assert True is False
E   +   where True = is_balanced('(')
```

Read the test ID in the FAILED line: the input in square brackets tells you which of the three checks is missing.

Part 4 — `evaluate_postfix(tokens)`, in `dsa/translation.py`

4.1 The idea

Lecture 06, “Application 2: Postfix”. In postfix each operator comes **after** its two operands, so there are no parentheses and no precedence rules. One stack of values evaluates it:

- a **number**: push it;
- an **operator**: pop **two** values, apply the operator, push the result;
- at the **end**: exactly one value is left, and it is the answer.

Postfix comes before shunting-yard in this lab, as in the lecture, because it is the easier of the two and because, once it works, it can check every answer your `infix_to_postfix` gives.

4.2 The operand-order trap

For `8 3 -`, the first value popped is 3 and the second is 8, and the answer is `8 - 3`: **the second value popped is the left operand**. Written with names:

```
right = values.pop()    # popped first
left  = values.pop()    # popped second
```

Get it the other way round and `+` and `*` still work, because they commute; `-` and `/` silently give wrong answers. Name your variables `left` and `right`, not `a` and `b`, and the bug has nowhere to hide.

4.3 Draw it

Lecture 06 traced `3 4 2 * +`. Trace `8 3 - 2 -` the same way on paper, as a table: token, action, stack (top on the right).

Checkpoint 6. (a) What is `8 3 - 2 -`? (b) What would a version with the operands swapped give for the same tokens? (c) What does `evaluate_postfix(['8', '2', '/'])` return — and why does the test’s `== 4` still pass?

4.4 Write it

1. Add `from dsa.stack import Stack` to `dsa/translation.py` (at the top, or inside the function). `evaluate_postfix` uses your `Stack`, not a list.
2. Make an empty `Stack` of values.
3. For each token:
 - if it is one of `+` `-` `*` `/`: if fewer than two values are on the stack, raise `ValueError`; otherwise pop right, pop left, compute `left <op> right`, and push the result;
 - otherwise it is a number: convert the string and push it. Use `int` unless the token contains a decimal point, then `float` — so `['3']` gives 3.
4. After the loop, if the stack does not hold **exactly one** value, raise `ValueError`. Otherwise pop it and return it.

Hints:

- Division is `/`, true division: `7 2 /` is 3.5. Do **not** use `//`, which gives 3 — `test_evaluate_postfix_divide` is there to catch it.
- You do not need to handle division by zero yourself: Python's `/` raises `ZeroDivisionError`, which is exactly what the docstring asks for.
- Check the count **before** popping. Otherwise an operator with too few operands raises your stack's `IndexError`, not `ValueError`.

4.5 Test it

```
pytest tests/test_translation.py -v -k evaluate_postfix
```

That runs the 11 tests of this week: `test_evaluate_postfix` (4 cases), `test_evaluate_postfix_operand_order`, `test_evaluate_postfix_divides_exactly`, `test_evaluate_postfix_rejects_malformed_input` (4 cases: `['']`, `['3', '4']`, `['+']`, `['3', '+']`) and `test_evaluate_postfix_division_by_zero`.

It is the command Lecture 06's Homework 6 gives.

Do not shorten it to `-k postfix`. That selects **12** tests: the extra one, `test_postfix_pairs_with_the_stack`, calls `tokenize` — Week 15 — and fails with `NotImplementedError` until then, even with a perfect `evaluate_postfix`. It is not your bug this week. Part 6 does the same check with `split()` instead.

4.6 When it fails

Operands swapped — three tests fail, not one:

```
E AssertionError: assert -2 == 2
E   + where -2 = evaluate_postfix(['5', '3', '-'])

E AssertionError: assert 0.2857142857142857 == 3.5
E   + where 0.2857142857142857 = evaluate_postfix(['7', '2', '/'])
```

and, in `test_evaluate_postfix_division_by_zero`, `1 0 /` computes `0 / 1`:

```
E Failed: DID NOT RAISE ZeroDivisionError
```

No “fewer than two” check — `['+', '3', '+']` pop an empty stack:

```
E IndexError: pop from empty stack
```

The test wanted `ValueError: malformed input` is the caller’s mistake, and the docstring says which error reports it.

No “exactly one at the end” check — `['3', '4']` returns 4:

```
E Failed: DID NOT RAISE ValueError
```

and `[]` pops an empty stack at the end: `IndexError: pop from empty stack`.

Floor division — `//` instead of `/`:

```
E AssertionError: assert 3 == 3.5
E + where 3 = evaluate_postfix(['7', '2', '/'])
```

Part 5 — `infix_to_postfix(tokens)`: the shunting-yard algorithm

5.1 The idea

Lecture 06, “Application 3: Infix to Postfix”. Read the infix tokens left to right, with an **output list** and an **operator stack**:

1. **Number** → output.
2. **Operator** → first pop to the output every operator on the stack with **higher or equal precedence**, stopping at a `(`; then push this one.
3. `(` → push.
4. `)` → pop to the output until the `(`; discard the `(`.
5. **End** → pop everything left to the output.

Precedence: `*` and `/` above `+` and `-`. Numbers go straight through; operators wait on the siding until a later operator of lower or equal precedence, a `)` or the end of the input lets them out.

Read rule 2 to the end: “stopping at a `(`”. A `(` on the stack is **not an operator** — it has no precedence, and it is a barrier. Operators inside a pair of parentheses must never leave past the `(`; only rule 4 removes it.

5.2 Trace before you code

Do these on paper **before** you open the editor. Use three columns — token, output, stack (top on the right) — exactly like the two traces in the lecture.

8 - 3 - 2 checks the “equal precedence pops” rule:

Token	Output	Stack
8	8	
-	8	-
3	8 3	-
-	8 3 -	- ← the first - has equal precedence: it leaves first
2	8 3 - 2	-
end	8 3 - 2 -	

Evaluate it: 8 3 - is 5, then 5 2 - is 3 — (8 - 3) - 2, left associative, as arithmetic requires.

2 * 3 + 4 checks that a lower-precedence operator empties the higher ones first:

Token	Output	Stack
2	2	
*	2	*
3	2 3	*
+	2 3 *	+ ← * is higher: it leaves, then + goes on
4	2 3 * 4	+
end	2 3 * 4 +	

Checkpoint 7. Trace $(1 + 2) * (3 - 4) / 5$ as a table. What is the postfix, and what value does your `evaluate_postfix` give for it?

5.3 Write it

1. A precedence table: a small dictionary `{'+': 1, '-': 1, '*': 2, '/': 2}`.
2. `output` is an empty Python list (it is the result); `operators` is an empty `Stack`.
3. For each token:
 - **operator**: while the stack is not empty, **and** its top is an operator (not `()`), **and** the top's precedence is `>=` this token's, pop the top to `output`. Then push the token.
 - `(`: push it.
 - `)`: while the top is not `(`, pop to `output`. Then pop the `(` and throw it away.
 - **anything else** is a number: append it to `output`.
4. At the end, pop everything left on the stack to `output`, and return `output`.

Every token is pushed at most once and popped at most once: $O(n)$.

Checkpoint 8. A student writes `>` instead of `>=` in step 3. What does their function return for `8 - 3 - 2`? What does that postfix evaluate to? Would `test_infix_to_postfix` alone catch it?

5.4 Test it

```
pytest tests/test_stack_queue.py -v -k infix
```

Two tests. `test_infix_to_postfix` has the two cases traced in the lecture, $3 + 4 * 2$ and $(3 + 4) * 2$. `test_infix_to_postfix_is_left_associative` has $8 - 3 - 2$ and $8 / 4 / 2$ — the “equal precedence pops” rule, which the first test cannot see (Checkpoint 8). Check your trace of Checkpoint 7 at the REPL too; no test has two pairs of parentheses.

5.5 When it fails

The (is treated as an operator — the “is it an operator?” test is missing from the while condition, so the precedence table is asked about (:

```
E   KeyError: '('
```

It happens in the second case, when `+` arrives with `(` on top of the stack.

> **instead of** `>=` — equal precedence stays on the stack, and the operators group from the right:

```
E   AssertionError: assert ['8', '3', '2', '-', '-'] == ['8', '3', '-', '2', '-']
E       At index 2 diff: '2' != '-'
```

The failing test is `test_infix_to_postfix_is_left_associative`; the other one still passes.

Nothing flushed at the end — the operators are still on the siding, and both tests fail. The first says:

```
E   AssertionError: assert ['3', '4', '2'] == ['3', '4', '2', '*', '+']
```

The (is not discarded — rule 4 pops to the `(` but leaves it there, and the end-of-input flush puts it into the output:

```
E   AssertionError: assert ['3', '4', '+', '2', '*', '('] == ['3', '4', '+', '2', '*']
E       Left contains one more item: '('
```

Postfix never contains a parenthesis. If one appears in your output, rule 4 is wrong.

Part 6 — The whole calculator

Lecture 06: “Infix $\rightarrow$ (shunting-yard) $\rightarrow$ postfix $\rightarrow$ (evaluation) $\rightarrow$ a value: two passes, two stacks, $O(n)$, no tree and no recursion.” You have both halves. Join them:

```
from dsa.stack import infix_to_postfix
from dsa.translation import evaluate_postfix

def calc(expression):
    tokens = expression.split()          # tokens must be separated by spaces
    postfix = infix_to_postfix(tokens)
    return postfix, evaluate_postfix(postfix)
```

```
for e in ['3 + 4 * 2', '( 3 + 4 ) * 2', '8 - 3 - 2',
         '2 * ( 3 + ( 4 - 1 ) )', '10 / 4']:
    print(e, '->', *calc(e))
```

```
3 + 4 * 2 -> ['3', '4', '2', '*', '+'] 11
( 3 + 4 ) * 2 -> ['3', '4', '+', '2', '*'] 14
8 - 3 - 2 -> ['8', '3', '-', '2', '-'] 3
2 * ( 3 + ( 4 - 1 ) ) -> ['2', '3', '4', '1', '-', '+', '*'] 12
10 / 4 -> ['10', '4', '/'] 2.5
```

This is what `test_postfix_pairs_with_the_stack_exercise` checks, with `split()` standing in for the Week 15 `tokenize`.

Checkpoint 9. Predict: (a) `calc('7 / 2 * 2')`; (b) `calc('3+4')` — written without spaces.

Checkpoint 9(b) is why Week 15 begins with a **tokenizer**: turning `'3+4'` or `'12*(3+4)'` into tokens is a job of its own. Week 15 then builds the other route from infix to a value — a recursive parser and an expression tree — and a test there checks that both routes agree.

Part 7 — Exercises at a glance

Method	Target cost	The trap	Tests
<code>Stack.push(value)</code>	$O(1)$ amortised	the top at the end of <code>self._items</code> — index 0 passes the tests and is $O(n)$	-k <code>lifo</code>
<code>Stack.pop()</code>	$O(1)$	pop the same end you push; <code>IndexError</code> when empty, not <code>None</code>	-k <code>"lifo or empty_behaviour"</code>
<code>Stack.peak()</code>	$O(1)$	read, do not remove; index <code>len - 1</code> , not <code>len</code> ; <code>IndexError</code> when empty	-k <code>"peek or empty_behaviour"</code>
<code>is_balanced(text)</code>	$O(n)$	three checks: partner, non-empty, empty at the end	-k <code>balanced (7)</code>
<code>evaluate_postfix(tokens)</code>	$O(n)$	second popped is the left operand; <code>ValueError</code> for malformed input; / not //	tests/test_translation.py -k <code>evaluate_postfix (11)</code>

Method	Target cost	The trap	Tests
<code>infix_to_postfix(tokens)</code>	$O(n)$	pop on higher or equal ; (stops the popping; discard the (; flush at the end	-k infix (2)

All 23 together:

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or empty_behaviour or balanced or infix_to_postfix"
pytest tests/test_translation.py -v -k evaluate_postfix
```

Worked solutions are in `solutions/dsa/stack.py` and `solutions/dsa/translation.py` — for **after** your tests pass, to compare, as `solutions/README.md` explains. `pytest --solutions ...` runs the same tests on them if you are unsure whether a failure is in your code or in your reading of the test.

Part 8 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week06-questions.md`, with answers in `week06-answers.md`.

1. **Write the code** — `practice/week06.py`, problems W6-C1 to W6-C5, checked by `pytest tests/test_practice_week06.py -v`. Use your Stack as the working storage in each. Start with **W6-C1** `reverse_words` (a warm-up), then **W6-C4** `tags_balanced` — this lab's Part 3 with tags instead of single characters — and **W6-C2** `MinStack`, whose hint is a second stack. **W6-C3** `next_greater` is the hardest and the most instructive: a loop inside a loop that is still $O(n)$ in total, because each index is pushed once and popped at most once — the same argument as shunting-yard.
2. **Trace** — on paper, then check at the REPL:
 - **W6-T1**: a sequence of push, pop, peek and `len` — the Checkpoint 2 kind of question, as the exam asks it;
 - **W6-T2**: evaluate `5 1 2 + 4 * + 3 -`, showing the stack after every token;
 - **W6-T5**: summing a nested list with a stack instead of recursion — in what order are the numbers printed, and why?
3. **Lecture 06, Homework 6, item 4**: sketch undo and redo for a text editor with two stacks. What happens to the redo stack when the user types after undoing?

Solutions: `solutions/practice/week06.py`, again for after.

Part 9 — Bridge to next week's lecture: queues

A stack touches **one** end, which is why either of your structures could keep its contract in $O(1)$. Lecture 07's queue is first in, first out: items join at the back and leave from the **front**. It needs

both ends — and on your `DynamicArray`, the front is exactly where `FrontStack` lost its race in Part 2. `pop(0)` shifts everything.

Here is one small idea to try now, with nothing but this week's `Stack`. Push three people on one stack, then pour it, item by item, into a second:

```
from dsa.stack import Stack

inbox, outbox = Stack(), Stack()
for person in ['Ali', 'Mona', 'Omar']:           # arrive in this order
    inbox.push(person)
while not inbox.is_empty():                       # pour: pop one, push the other
    outbox.push(inbox.pop())
print(outbox)
print(outbox.pop(), outbox.pop(), outbox.pop())

Stack(['Omar', 'Mona', 'Ali']) # top = 'Ali'
Ali Mona Omar
```

Pouring reversed the order, and LIFO reversed is FIFO: they leave in the order they arrived. Lecture 07 turns this into a whole queue — and asks what it costs when people keep arriving while others leave. Before the lecture, think about one question: in your `DynamicArray`, what would it take to remove from the front **without** shifting anything?

Summary

Idea	The one line to keep
The Stack ADT	LIFO: push, pop, peek act on the top only, all $O(1)$.
Empty stack	pop and peek raise <code>IndexError</code> — your own, with a stack message.
Where the top goes	The end of the <code>DynamicArray</code> : append and pop there move nothing.
The dishonest stack	Top at index 0: same values, same tests pass, $O(n)$ per operation — measured quadratic.
Balanced brackets	Push openers; each closer needs a non-empty stack and its partner on top; empty at the end.
Postfix	Push numbers; an operator pops two — the second popped is the left operand .
Shunting-yard	Numbers to the output; operators wait on a stack and leave on higher or equal precedence, at <code>)</code> , or at the end.
The calculator	Infix $\rightarrow$ postfix $\rightarrow$ value: two passes, two stacks, $O(n)$.

Idea	The one line to keep
The -k filter	-k also matches the file name: use the test names, not stack.

Answers to the checkpoints

Checkpoint 1. `Stack()` gives `Stack([])`, `len(Stack())` is `0` and `Stack().is_empty()` is `True`: all three use only given code. `Stack([1, 2])` raises `NotImplementedError`, because the given `__init__` calls **your** `push` once for each value — so even building a stack from a list needs Part 1.

Checkpoint 2.

```
c
d
3
Stack(['a', 'b', 'd']) # top = 'd'
```

`pop` removed 'c', the last in; 'd' went on top of 'b'; `peek` did not remove it, so three items are left.

Checkpoint 3. It raises `IndexError: -1 — from DynamicArray.__getitem__`, because `index` `len - 1` is `-1` on an empty stack, and `-1 + 0` is still out of range. The test only checks the **type** of the error, so it passes. Write the check anyway: the message `-1` tells the caller nothing about stacks, a missing check in `pop` would report `pop from empty DynamicArray` — the implementation leaking through the ADT — and the stack's contract should not depend on last week's error messages.

Checkpoint 4. (a) Yes: all three stack tests pass. `FrontStack` returns the same values in the same order and raises the same errors; only its cost is wrong. (b) The top-at-the-end times roughly **double** ($O(n)$ for n operations); the top-at-index-0 times roughly **quadruple** ($O(n^2)$ for n operations).

Checkpoint 5. `"a)(b"` is `False` — the `)` arrives with nothing open (the non-empty check). `"(]"` is `False` — `]` pops `(` (the partner check). `"((a)"` is `False` — one `(` is left at the end (the empty-at-the-end check). `"{[( )]}x"` is `True` — every closer matches, and the `x` is ignored. `"}{"` is `False` — the `}` arrives with the stack empty (the non-empty check); the function returns before it ever sees the `{`.

Checkpoint 6. (a) `3: 8 3 -` is 5, then `5 2 -` is 3. (b) 7: the swapped version computes `3 - 8 = -5`, then `2 - (-5) = 7`. (c) `4.0`, a float, because `/` is true division and always gives a float; the test's `== 4` passes because `4.0 == 4` is `True` in Python.

Checkpoint 7. The postfix is `1 2 + 3 4 - * 5 /`, and it evaluates to `-0.6` (3 times `-1` is `-3`, divided by 5).

Token	Output	Stack
(		(
1	1	(
+	1	(+
2	1 2	(+
)	1 2 +	
*	1 2 +	*
(	1 2 +	* (
3	1 2 + 3	* (
-	1 2 + 3	* (- ← (stops the popping: * stays
4	1 2 + 3 4	* (-
)	1 2 + 3 4 -	*
/	1 2 + 3 4 - *	/ ← * has equal precedence: it leaves
5	1 2 + 3 4 - * 5	/
end	1 2 + 3 4 - * 5 /	

Checkpoint 8. ['8', '3', '2', '-', '-'], which evaluates to $8 - (3 - 2) = 7$ instead of 3: with `>`, the first `-` stays on the stack when the second arrives, so the two group from the right. `test_infix_to_postfix` alone would **not** catch it — it passes — because neither of its two cases has two operators of equal precedence in a row. That is why the second test, `test_infix_to_postfix_is_left_associative`, exists: it fails with `assert ['8', '3', '2', '-', '-'] == ['8', '3', '-', '2', '-']`.

Checkpoint 9. (a) (['7', '2', '/', '2', '*'], 7.0): `/` and `*` have equal precedence, so they apply left to right, $(7 / 2) * 2$, and the result is a float because of the `/`. (b) `'3+4'.split()` is `['3+4']` — **one** token. `infix_to_postfix` passes it to the output as if it were a number, and `evaluate_postfix` fails to convert it: `ValueError: invalid literal for int() with base 10: '3+4'`. Splitting on spaces is not tokenizing; Week 15's `tokenize` is.